

Documentation Technique

Backend Complet

Marché-IA Cameroun — Plateforme IA Marchés Publics

42 fichiers · 4 264 lignes de code · Python / FastAPI / PostgreSQL / Claude AI

Version 1.0 — Avril 2026

Document confidentiel — Usage interne équipe projet

**24 fichiers
Python**

6 fichiers SQL

**32 endpoints
API**

10 tables BDD

Table des matières

Partie 1	Vue d'ensemble — Ce qu'est ce backend
Partie 2	Architecture — Les 6 couches du système
Partie 3	La base de données — 10 tables en détail
Partie 4	Les modèles SQLAlchemy — Chaque table expliquée
Partie 5	Les schémas Pydantic — Validation des données
Partie 6	Les 8 services — La logique métier
Partie 7	Le moteur IA RAG — Comment l'IA répond
Partie 8	Les 32 endpoints API — Toutes les routes
Partie 9	Le worker Celery — Les tâches automatiques
Partie 10	Sécurité — JWT, bcrypt, permissions
Partie 11	Comment posséder ce backend — Guide complet
Partie 12	Variables d'environnement — Configuration
Partie 13	Commandes de diagnostic et maintenance

Qu'est-ce que ce backend ?

Ce backend est le cerveau invisible de la plateforme Marché-IA Cameroun. C'est un serveur API développé en Python avec le framework FastAPI. Il reçoit toutes les requêtes du site web, les traite, interroge la base de données, appelle l'intelligence artificielle Claude, et renvoie les réponses formatées.

En termes simples : quand un utilisateur pose une question sur les marchés publics, c'est ce backend qui cherche les bons articles de loi, les envoie à Claude, reçoit la réponse et la renvoie à l'écran. Quand une alerte est configurée, c'est ce backend qui scrute l'ARMP toutes les heures et envoie les notifications WhatsApp.

Ce qu'il accomplit quotidiennement

1. Répondre aux questions juridiques

Le RAG Service cherche les articles pertinents du Code des marchés 2018 dans Qdrant, les transmet à Claude, et retourne une réponse avec les sources citées en moins de 3 secondes.

2. Authentifier et sécuriser

Crée les comptes, vérifie les mots de passe avec bcrypt, génère des tokens JWT signés HS256 valables 24 heures, et protège chaque endpoint sensible.

3. Synchroniser l'ARMP automatiquement

Toutes les heures, le worker Celery récupère les nouveaux avis d'appels d'offres sur armp.cm (RSS d'abord, scraping HTML en fallback) et les stocke en base.

4. Envoyer des alertes personnalisées

20 minutes après chaque sync, le worker matche les nouveaux marchés avec les alertes actives et notifie par email HTML ou WhatsApp — avec dédoublement garanti.

5. Générer des documents officiels

Sur demande, appelle Claude avec un prompt juridique spécialisé pour produire lettres de soumission, offres techniques, recours ARMP, projets de marché.

6. Fournir les statistiques publiques

Calcule en temps réel les agrégats SQL pour le tableau de bord de transparence (marchés par région, secteur, autorité) — sans authentification requise.

PARTIE 2 — ARCHITECTURE — LES 6 COUCHES

Le backend est organisé en couches strictement séparées. Chaque couche a une responsabilité unique. Une couche ne peut pas 'sauter' une autre couche.

Couche	Rôle précis
Couche 1 — Clients	Frontend Next.js, mobile, WhatsApp bot, partenaires via API. Envoyent des requêtes HTTP.
Couche 2 — API Gateway	FastAPI reçoit toutes les requêtes. Vérifie le token JWT, applique CORS, mesure le temps de traitement, gère les erreurs.
Couche 3 — Router (router.py)	Dispatche chaque URL vers la bonne fonction endpoint. Valide les données entrantes avec Pydantic.
Couche 4 — Services	8 classes de services contenant toute la logique métier : UserService, TenderService, RAGService, AlertService, etc.
Couche 5 — Modèles	9 classes SQLAlchemy qui représentent les tables PostgreSQL. Tout accès base de données passe par eux.
Couche 6 — Données	PostgreSQL (données structurées) + Qdrant (vecteurs IA) + Redis (cache réponses) + fichiers knowledge_base (PDFs).

Flux d'une requête — de l'utilisateur à la réponse

Voici exactement ce qui se passe quand un utilisateur pose la question « Quel est le délai de recours après une attribution ? »

- Étape 1** — Le navigateur envoie POST /api/v1/chat avec le token JWT et la question en JSON
- Étape 2** — FastAPI vérifie le token JWT (decode_token dans security.py)
- Étape 3** — Le router.py appelle la fonction ask() dans le router
- Étape 4** — ask() appelle ConversationService pour charger l'historique
- Étape 5** — ask() appelle rag_service.chat() — le moteur IA
- Étape 6** — RAGService encode la question en vecteur (768 dimensions)
- Étape 7** — Qdrant retourne les 8 articles de loi les plus proches sémantiquement
- Étape 8** — Le contexte juridique + la question sont envoyés à Claude API
- Étape 9** — Claude génère la réponse en citant l'Article 74 du Code des marchés
- Étape 10** — La réponse + sources sont sauvegardées en base via ConversationService
- Étape 11** — FastAPI retourne le JSON au navigateur : réponse, sources, tokens utilisés, latence

PARTIE 3 — LA BASE DE DONNÉES — 10 TABLES

La base PostgreSQL contient 10 tables, 30+ index, 4 vues, 5 fonctions PL/pgSQL et 20+ contraintes CHECK. Les migrations s'exécutent automatiquement au démarrage Docker.

Table	Description détaillée
users	Comptes utilisateurs. Rôles : citizen company authority institution admin. Email unique, mot de passe haché bcrypt, secteurs en tableau PostgreSQL.
tenders	Appels d'offres ARMP. Synchronisés automatiquement. Index full-text GIN sur titre et référence pour la recherche rapide. Index partiel sur deadline WHERE status='open'.
conversations	Sessions de chat IA. Titre auto-généré à partir du 1er message. Status : active archived deleted. Lié à users par FK avec CASCADE.
messages	Messages individuels. Stocke question + réponse + sources JSON + tokens_input/output + latency_ms + feedback (-1/0/1). Index sur conversation_id + created_at.
alerts	Préférences de veille. secteurs et régions stockés en TEXT[] (tableaux PostgreSQL natifs). Contrainte CHECK sur channel et sur la cohérence min/max_amount.
alert_matches	Table de jonction alertes/marchés. Contrainte UNIQUE (alert_id, tender_id) garantit l'absence de doublon. Index partiel WHERE notified=FALSE pour le worker.
documents	Documents générés par IA. 6 types contraints par CHECK. Version incrémentée à chaque régénération. Metadata JSON stocke les paramètres de génération.
knowledge_chunks	Index textuel des chunks Qdrant. Permet recherche full-text SQL en complément. Index GIN sur to_tsvector('french', content).
audit_logs	Traçabilité totale. Toutes les actions importantes avec IP, user_agent, horodatage. Purgé après 90 jours par le worker nocturne.
sync_jobs	Historique des syncs ARMP. status : running success failed. Permet de diagnostiquer les problèmes de synchronisation.

Les 4 vues SQL

Vue	Rôle
v_dashboard_public	Agrège les marchés par secteur, région, procédure et statut. Alimente le tableau de bord public sans authentification.
v_tenders_urgent	Marchés ouverts dont la deadline est dans 7 jours ou moins. Calcule days_remaining = deadline - CURRENT_DATE.
v_sync_health	Taux de succès des syncs ARMP sur 30 jours. Affiche success_rate_pct pour le monitoring admin.

v_ai_usage

Métriques IA par jour : tokens consommés, latence moyenne, taux de satisfaction (feedback positifs / total notés).

PARTIE 4 — LES MODÈLES SQLALCHEMY

Fichier : `app/models/models.py` — Contient toutes les classes Python qui correspondent aux tables de la base de données. SQLAlchemy traduit automatiquement les opérations Python en requêtes SQL.

PRINCIPE CLÉ : On n'écrit jamais de SQL brut dans le code applicatif. On utilise les modèles SQLAlchemy qui génèrent le SQL optimal automatiquement. Cela protège contre les injections SQL et rend le code lisible.

class User(Base)

→ Table PostgreSQL : *users*

- `id` : UUID — généré automatiquement par `uuid_generate_v4()`
- `email` : Text, UNIQUE, `index=True` — indexé pour connexion rapide
- `hashed_password` : Text — hash bcrypt, JAMAIS le mot de passe en clair
- `role` : String(50), `default='citizen'` — citizen|company|authority|institution|admin
- `sectors` : ARRAY(Text) — ex: ['travaux', 'fournitures', 'IT']
- `preferences` : JSONB — stockage flexible (langue, notifications...)
- Relation conversations → CASCADE (supprimer user supprime ses conversations)
- Relation alerts → CASCADE
- Relation documents → CASCADE

class Tender(Base)

→ Table PostgreSQL : *tenders*

- `external_id` : Text, UNIQUE — ID source ARMP pour éviter les doublons lors des syncs
- `reference` : Text — ex: 007/AONO/MINEE/2026 (référence officielle)
- `procedure_type` : String(50) — AONO|AOIO|AOR|DC|gre_a_gre|AMI|DDP
- `estimated_amount` : BigInteger — montant en FCFA (peut dépasser 2 milliards)
- `deadline` : Date, `index=True` — indexée car très souvent filtrée
- `status` : String(50), `index=True` — open|closed|awarded|cancelled|infructuous
- `raw_data` : JSONB — données brutes ARMP conservées pour traçabilité
- `synced_at` : DateTime — horodatage de la dernière synchronisation

class Conversation + Message

→ Table PostgreSQL : *conversations + messages*

- `Conversation.title` auto-généré : 60 premiers caractères du 1er message
- `Conversation.status` : active|archived|deleted (archive = soft delete)
- `Message.role` : user|assistant|system
- `Message.sources` : JSONB — [{rank, source_name, article_ref, score, source_type}]
- `Message.tokens_input` + `tokens_output` — suivi coût Claude API
- `Message.latency_ms` — performance monitoring
- `Message.feedback` : SmallInteger — -1 mauvais | 0 neutre | 1 bon
- Relation conversation → messages : `order_by created_at ASC`

class Alert + AlertMatch

→ Table PostgreSQL : **alerts + alert_matches**

- Alert.sectors : ARRAY(Text) — secteurs à surveiller
- Alert.keywords : ARRAY(Text) — mots-clés dans les titres de marchés
- Alert.channel : String(50) — in_app|email|whatsapp|all
- Alert.active : Boolean, index=True — index partiel WHERE active=TRUE
- Alert.last_fired — affiché 'Dernière alerte il y a X heures'
- AlertMatch.UNIQUE(alert_id, tender_id) — garantit zéro doublon de notification
- AlertMatch.notified : Boolean — index partiel WHERE notified=FALSE
- AlertMatch.score : Float — pertinence du matching (0.0 à 1.0)

class Document

→ Table PostgreSQL : **documents**

- type : String(100) —
submission_letter|technical_offer|financial_offer|qualification_file|recourse|contract_draft
- content : Text — texte complet généré par Claude (peut faire plusieurs pages)
- metadata_ (colonne 'metadata') : JSONB — company_name, representative, tokens, generated_at
- status : String(50) — draft|finalized|submitted
- version : Integer, default=1 — incrémenté à chaque régénération
- tender_id : FK optional → SET NULL si le marché est supprimé

class AuditLog + SyncJob

→ Table PostgreSQL : **audit_logs + sync_jobs**

- AuditLog.action — format 'entité.action' ex: auth.login, chat.message, doc.generate
- AuditLog.entity_type + entity_id — référence optionnelle vers l'objet concerné
- AuditLog.ip_address — pour détecter les connexions suspectes
- SyncJob.source — armp_feed|coleps_api|manual
- SyncJob.status — running|success|failed
- SyncJob.records_synced — nombre de nouveaux marchés créés
- Purge automatique après 90 jours par le worker nightly_cleanup

PARTIE 5 — LES SCHÉMAS PYDANTIC

Fichier : `app/schemas/schemas.py` — Les schémas Pydantic sont les gardiens de l'entrée. Avant qu'une donnée entre dans le système, elle passe par un schéma qui vérifie qu'elle est correcte. Une donnée invalide est rejetée immédiatement avec un message d'erreur clair.

Schéma	Ce qu'il valide
RegisterRequest	Valide l'inscription : email format correct, mot de passe 8+ caractères, role dans la liste autorisée, secteurs en liste.
LoginRequest	Email + mot de passe. Pydantic valide le format email.
TokenResponse	Retourne <code>access_token</code> + <code>token_type='bearer'</code> + <code>expires_in</code> en secondes.
UserOut	Profil utilisateur retourné à l'API. Ne contient JAMAIS le <code>hashed_password</code> .
ChatRequest	question entre 3 et 2000 caractères (<code>min_length/max_length</code>). <code>conversation_id</code> optionnel (UUID). <code>filters</code> optionnel (dict).
ChatResponse	Réponse complète : <code>answer</code> , <code>sources</code> (liste de <code>RAGSource</code>), <code>tokens_used</code> , <code>latency_ms</code> , <code>from_cache</code> .
RAGSource	Source juridique citée : <code>rank</code> , <code>source_name</code> , <code>article_ref</code> , <code>content_snippet</code> , <code>score</code> (0-1), <code>source_type</code> .
TenderListResponse	Réponse paginée : <code>items</code> (liste <code>TenderOut</code>), <code>total</code> , <code>page</code> , <code>pages</code> , <code>limit</code> . Chaque <code>TenderOut</code> inclut <code>days_until_deadline</code> calculé.
AlertCreate	Création d'alerte avec validation : <code>channel</code> doit être <code>in_app email whatsapp all</code> .
DocumentGenerateRequest	Valide les 6 types de documents. <code>company_name</code> et <code>representative</code> sont obligatoires.
DashboardFull	Réponse complète du dashboard : <code>stats</code> , <code>by_month</code> , <code>by_sector</code> , <code>by_region</code> , <code>by_procedure</code> , <code>top_authorities</code> .

PARTIE 6 — LES 8 SERVICES — LA LOGIQUE MÉTIER

Fichier : `app/services/services.py` — Les services contiennent toute la logique métier. Les endpoints appellent les services. Les services utilisent les modèles SQLAlchemy. Cette séparation garantit que la logique est testable et réutilisable.

UserService

- `get_by_id(user_id)` → cherche par UUID, retourne None si absent
- `get_by_email(email)` → normalise l'email en minuscules avant la recherche
- `create(data)` → insère un nouvel utilisateur, retourne l'objet créé avec son UUID
- `update(user_id, data)` → met à jour uniquement les champs non-None (patch partiel)
- `update_last_login(user_id)` → enregistre l'horodatage de connexion

ConversationService

- `get_or_create(user_id, conv_id)` → charge ou crée une conversation
- `get_history(conv_id, limit=10)` → retourne les N derniers messages pour le contexte IA
- `_auto_title(conv_id, first_msg)` → titre auto sur le 1er message (60 caractères)
- `save_message(...)` → sauvegarde question ou réponse avec toutes les métadonnées
- `update_feedback(msg_id, score)` → enregistre la note utilisateur (-1/0/1)
- `archive(conv_id)` → soft delete : `status='archived'`, données conservées

TenderService

- `list(...)` → requête SQL dynamique selon 8 critères de filtre + pagination
- `get_by_id(tender_id)` → retourne le marché ou None
- `upsert_tender(data)` → insert si nouveau, update si `external_id` connu. Retourne True si nouveau.
- `close_expired()` → `UPDATE tenders SET status='closed' WHERE deadline < CURRENT_DATE AND status='open'`
- `create_sync_job` / `complete_sync_job` / `fail_sync_job` → cycle de vie d'une synchronisation ARMP

AlertService

- `create(user_id, data)` → crée une alerte avec les critères
- `find_matches(alert, since)` → requête SQL dynamique avec les critères de l'alerte depuis une date
- `is_notified(alert_id, tender_id)` → vérifie dans `alert_matches` si déjà notifié
- `mark_notified(alert_id, tender_id)` → insère dans `alert_matches` + met à jour `last_fired`
- `count_preview(user_id, data)` → simule le matching sur 30 jours sans sauvegarder

DocumentService

- `generate(user_id, req)` → choisit le prompt selon le type, appelle Claude à `temperature=0.2`
- `_build_prompt(req)` → assemble toutes les infos entreprise + marché en prompt structuré
- 6 system prompts distincts → un par type de document, avec les clauses obligatoires
- `update` / `delete` → modification ou suppression d'un document généré

NotificationService

- `send(alert, tender, channel)` → dispatche selon email|whatsapp|in_app|all
- `_email()` → MIMEMultipart HTML + texte brut, SMTP TLS, en-tête de marque
- `_whatsapp()` → Meta Cloud API v19.0, message formaté avec gras et émojis
- Gestion d'erreur silencieuse → log l'erreur sans planter le worker

DashboardService

- `get_full()` → appelle les 5 méthodes agrégées en parallèle
- `get_stats()` → COUNT + SUM + AVG en une seule requête SQL, filtré sur l'année courante
- `_by_month()` → GROUP BY to_char(publication_date, 'YYYY-MM') sur 12 mois
- `_by_sector()` → GROUP BY sector avec calcul de pourcentage
- `_top_authorities()` → TOP 10 par COUNT(*) DESC LIMIT 10

RAGService

- `initialize()` → charge SentenceTransformer, ouvre connexions Qdrant + Anthropic + Redis
- `chat(question, history, user_id, filters)` → pipeline 7 étapes (voir Partie 7)
- `chat_stream(question, history, filters)` → AsyncGenerator SSE token par token
- `_cache_key(question, filters)` → hash SHA-256 des 16 premiers caractères
- `close()` → ferme proprement toutes les connexions au shutdown FastAPI

PARTIE 7 — LE MOTEUR IA RAG — COMMENT L'IA RÉPOND

RAG signifie Retrieval-Augmented Generation. C'est la technologie qui permet à l'IA de répondre précisément en s'appuyant sur les textes officiels camerounais plutôt que sur ses données d'entraînement générales.

Le pipeline en 7 étapes (déclenchées à chaque question)

Étape 1 — Clé de cache

Génère un hash SHA-256 de la question normalisée. Vérifie Redis. Si présent → retourne la réponse en 5ms, zéro appel IA, économie de 60-70% sur les coûts Claude.

Étape 2 — Embedding

Encode la question en vecteur de 768 dimensions via paraphrase-multilingual-mpnet-base-v2. Ce modèle comprend le français, l'anglais et 50+ langues. La question devient une suite de 768 nombres flottants.

Étape 3 — Recherche Qdrant

Envoie le vecteur à Qdrant qui calcule la similarité cosinus avec les milliers de chunks du Code des marchés. Retourne les 8 chunks avec score > 0.65. En moins de 50ms.

Étape 4 — Contexte

Assemble les 8 chunks en un bloc de texte structuré avec leurs références (Article 48, Section III.9...) et scores. C'est le contexte juridique injecté dans Claude.

Étape 5 — Prompt Claude

Construit le message final : system prompt expert marchés publics + contexte juridique + historique des 10 derniers messages + question. Température : 0.1 (très factuel).

Étape 6 — Génération

Appelle l'API Anthropic (claude-sonnet-4-6). En mode normal : attend la réponse complète. En mode stream : reçoit et transmet les tokens au fur et à mesure via SSE.

Étape 7 — Cache + retour

Stocke la réponse dans Redis pour 1 heure (TTL). Retourne : answer, sources (liste de RAGSource), tokens_input, tokens_output, latency_ms, from_cache=False.

Le system prompt — Instructions données à Claude

Claude reçoit à chaque appel un system prompt qui lui donne son rôle et ses règles :

```
Tu es un expert spécialisé dans le Code des marchés publics camerounais
(Décret n°2018/366 du 20 juin 2018 et textes d'application).
```

```
RÈGLES ABSOLUES :
```

1. Cite toujours l'article ou le texte exact sur lequel tu te bases.
2. Si une information n'est pas dans le contexte fourni, dis-le clairement
– n'invente JAMAIS.
3. Réponds en français par défaut, en anglais si l'utilisateur écrit en anglais.

4. Donne des réponses pratiques et actionnables, pas seulement théoriques.
5. Signale toujours les délais légaux quand ils sont pertinents.
6. Pour les recours, précise les délais et la procédure ARMP (Article 74).

Pourquoi le cache réduit les coûts de 60-70% ?

Les questions les plus fréquentes sur une plateforme marchés publics sont toujours les mêmes : 'Comment soumettre une offre ?', 'Quel est le délai de recours ?', 'Quelles sont les conditions d'un gré à gré ?'. Redis mémorise ces réponses pendant 1 heure. Si 100 utilisateurs posent la même question dans l'heure, Claude n'est appelé qu'une seule fois. Les 99 autres fois, la réponse arrive en 5 millisecondes.

PARTIE 8 — LES 32 ENDPOINTS API

Fichier : `app/api/v1/router.py` — Les endpoints sont les URLs que le frontend appelle. Chaque endpoint reçoit une requête HTTP, valide les données (Pydantic), appelle le bon service, et retourne une réponse JSON.

Authentification — `/api/v1/auth/` (5 endpoints)

Endpoint	Accès	Ce qu'il fait
POST <code>/auth/register</code>	Public	Crée un compte. Vérifie email unique, hache le mot de passe, trace dans <code>audit_logs</code> , retourne token JWT.
POST <code>/auth/login</code>	Public	Vérifie email + mot de passe bcrypt. Enregistre <code>last_login</code> . Retourne token JWT 24h.
GET <code>/auth/me</code>	JWT requis	Retourne le profil complet de l'utilisateur connecté. NE retourne JAMAIS <code>hashed_password</code> .
PATCH <code>/auth/me</code>	JWT requis	Met à jour uniquement les champs fournis (patch partiel). Ignore les champs <code>None</code> .
POST <code>/auth/change-password</code>	JWT requis	Vérifie ancien mdp, stocke nouveau hash. Retourne 204 (pas de contenu).

Assistant IA — `/api/v1/chat/` (5 endpoints)

Endpoint	Accès	Ce qu'il fait
POST <code>/chat</code>	JWT requis	Appelle le pipeline RAG complet. Sauvegarde question + réponse en base. Retourne réponse + sources + métriques.
POST <code>/chat/stream</code>	JWT requis	Même pipeline mais en Server-Sent Events. Le frontend reçoit d'abord les sources, puis les tokens un par un.
GET <code>/chat/history</code>	JWT requis	Liste les conversations actives de l'utilisateur, triées par <code>updated_at</code> DESC. Paginé.
DELETE <code>/chat/history/{id}</code>	JWT requis	Archive une conversation (soft delete). Les données restent en base pour l'audit.
POST <code>/chat/feedback/{id}</code>	JWT requis	Enregistre -1 0 1 sur un message. Vérifie que le message appartient à l'utilisateur.

Appels d'offres — `/api/v1/tenders/` (3 endpoints publics)

Endpoint	Accès	Ce qu'il fait
GET <code>/tenders</code>	Public	8 filtres URL : <code>search</code> , <code>status</code> , <code>sector</code> , <code>region</code> , <code>procedure_type</code> , <code>min/max_amount</code> , <code>dates</code> . Pagination. Tri configurable.

GET /tenders/{id}	Public	Détail complet. Calcule days_until_deadline pour les marchés ouverts.
GET /tenders/export	Public	Génère un CSV StreamingResponse avec jusqu'à 1000 marchés filtrés.

Alertes — /api/v1/alerts/ (7 endpoints)

Endpoint	Accès	Ce qu'il fait
GET /alerts	JWT requis	Liste les alertes avec match_preview calculé en temps réel (nb marchés 30j).
POST /alerts	JWT requis	Crée une alerte, calcule le preview, trace dans audit_logs.
GET /alerts/preview	JWT requis	Simulation temps réel : combien de marchés correspondraient ?
PATCH /alerts/{id}	JWT requis	Modifie les critères. Patch partiel.
PATCH /alerts/{id}/toggle	JWT requis	Bascule active ↔ inactive en un appel.
DELETE /alerts/{id}	JWT requis	Suppression définitive avec trace audit.
GET /alerts/notifications	JWT requis	50 dernières notifications. Filtre unread_only disponible.

Documents IA — /api/v1/documents/ (7 endpoints)

Endpoint	Accès	Ce qu'il fait
GET /documents/types	Public	Liste les 6 types de documents avec leurs libellés en français.
GET /documents	JWT requis	Mes documents triés par date de création DESC.
POST /documents	JWT requis	Génère un document par IA (Claude). Valide le type, appelle DocumentService.generate().
GET /documents/{id}	JWT requis	Détail d'un document avec le contenu complet.
PATCH /documents/{id}	JWT requis	Modifie le contenu ou le statut (draft→finalized→submitted).
DELETE /documents/{id}	JWT requis	Suppression définitive.
GET /documents/{id}/download	JWT requis	Télécharge le contenu en texte brut (StreamingResponse).

Dashboard transparence — /api/v1/dashboard/ (2 endpoints publics)

Endpoint	Accès	Ce qu'il fait
GET /dashboard	Public	Toutes les statistiques : stats globales + by_month + by_sector + by_region + by_procedure + top_authorities.

GET /dashboard/stats	Public	KPIs uniquement : totaux, montants, délais, dernière sync ARMP.
----------------------	--------	---

PARTIE 9 — LE WORKER CELERY — TÂCHES AUTOMATIQUES

Fichier : `app/workers/sync_worker.py` — Le worker Celery est un processus séparé qui tourne en permanence en parallèle du serveur API. Il exécute 4 tâches planifiées automatiquement, sans intervention humaine.

`sync_armp_feed`

Planification : Toutes les heures — minute 0

- Crée un SyncJob en base avec `status='running'`
- Tente de parser le flux RSS de l'ARMP (`feedparser`)
- Si RSS échoue → scraping HTML de `armp.cm` avec `BeautifulSoup`
- Extrait la référence officielle avec regex (pattern `XX/AONO/MINISTERE/2026`)
- Pour chaque avis : `upsert_tender()` → insert si nouveau, update si connu
- Met à jour le SyncJob : `status='success'`, `records_synced=N`
- En cas d'erreur : `status='failed'`, message d'erreur, retry x3 après 5 min

`match_and_notify`

Planification : Toutes les heures — minute 20

- Récupère toutes les alertes actives (`WHERE active=TRUE`)
- Pour chaque alerte : `find_matches()` avec les marchés des 2 dernières heures
- Pour chaque match : `is_notified()` — vérifie `alert_matches`
- Si non notifié : `send()` → email HTML ou WhatsApp selon le canal
- `mark_notified()` → insère dans `alert_matches` + met à jour `last_fired`
- Garantit zéro doublon grâce à la contrainte `UNIQUE` de `alert_matches`

`close_expired`

Planification : Toutes les heures — minute 40

- `UPDATE tenders SET status='closed'`
- `WHERE status='open' AND deadline < CURRENT_DATE AND deadline IS NOT NULL`
- Retourne le nombre de marchés fermés
- Appelé automatiquement — aucune intervention manuelle requise

`nightly_cleanup`

Planification : Chaque nuit à 3h00

- `DELETE FROM audit_logs WHERE created_at < NOW() - INTERVAL '90 days'`
- `DELETE FROM sync_jobs WHERE started_at < NOW() - INTERVAL '90 days'`
- Libère l'espace base de données sans supprimer les données récentes
- Journalise le nombre d'enregistrements supprimés

PARTIE 10 — SÉCURITÉ

Mots de passe — bcrypt

Aucun mot de passe n'est jamais stocké en clair. La fonction `hash_password()` utilise `bcrypt` avec un facteur de travail de 12 rounds. `bcrypt` génère un salt aléatoire différent à chaque hachage : deux utilisateurs avec le même mot de passe ont des hashes différents en base.

```
# security.py – bcrypt

hash_password('Azerty123!') → '$2b$12$ew8lFnzX...' (stocké en base)

verify_password('Azerty123!', hash) → True (connexion réussie)

verify_password('mauvais', hash) → False (connexion refusée)
```

Authentification — JWT HS256

Après connexion, le backend génère un token JWT signé avec la `SECRET_KEY`. Ce token contient l'UUID de l'utilisateur et expire après 24h. Le frontend l'envoie dans chaque requête via le header `Authorization: Bearer`. Le backend le decode et vérifie la signature à chaque appel.

```
# security.py – JWT

# Génération du token (connexion réussie)

token = create_access_token('user-uuid-abc123')

# → 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'

# Vérification (à chaque requête)

user_id = decode_token('eyJhbGciOiJIUzI1NiIs...')

# → 'user-uuid-abc123' ou None si invalide/expiré
```

Protection des endpoints — Depends(get_current_user)

Chaque endpoint protégé déclare `current_user=Depends(get_current_user)`. FastAPI exécute automatiquement `get_current_user()` avant le code de l'endpoint. Si le token est absent, invalide ou expiré → 401 Unauthorized. Si l'utilisateur est désactivé → 401. Sinon → l'utilisateur est passé au endpoint.

Données personnelles

Donnée	Traitement
Mot de passe	Jamais stocké en clair. Hash bcrypt uniquement.
Token JWT	Stocké côté client (localStorage/cookie). Expiré après 24h.
Adresse IP	Stockée dans <code>audit_logs</code> pour traçabilité sécurité. Purgée après 90 jours.
Données IA	Les questions posées à l'IA sont sauvegardées en base pour l'historique. Non utilisées pour entraîner Claude.

Données ARMP

raw_data JSONB conserve les données brutes pour traçabilité et débogage.

PARTIE 11 — COMMENT POSSÉDER CE BACKEND — GUIDE COMPLET

Ce guide explique pas à pas comment installer, démarrer, tester et maîtriser ce backend sur n'importe quelle machine. Durée estimée : 30 à 60 minutes.

Pré-requis à installer

Outil	Pourquoi et où l'obtenir
Docker Desktop	Télécharger sur docker.com — Lance toute l'infrastructure en une commande. OBLIGATOIRE.
Python 3.11+	Pour exécuter les scripts localement (ingest.py, tests). python.org
Git	Pour cloner le projet et gérer les versions. git-scm.com
Clé Claude API	Créer un compte sur console.anthropic.com → API Keys → Créer une clé. OBLIGATOIRE.
Éditeur de texte	VS Code recommandé avec l'extension Python. code.visualstudio.com

Installation et démarrage — 6 étapes

1

Décompresser le projet

Décompresser `marche-ia-backend-COMPLET.zip` dans un dossier de votre choix. Ouvrir un terminal dans ce dossier.

```
unzip marche-ia-backend-COMPLET.zip
cd backend-final
```

2

Créer le fichier de configuration

Copier le modèle de configuration et le remplir avec vos vraies valeurs.

```
cp .env.example .env

# Sur Mac/Linux : nano .env
# Sur Windows : notepad .env

# Remplir OBLIGATOIREMENT :

# SECRET_KEY=<résultat de : openssl rand -hex 32>
# ANTHROPIC_API_KEY=sk-ant-api03-VOTRE_CLE
```

3

Démarrer Docker Desktop

Ouvrir Docker Desktop et attendre qu'il soit prêt (icône stable dans la barre des tâches).

```
# Vérifier que Docker est actif :
```

```
docker info
```

4

Lancer toute l'infrastructure

Une seule commande démarre PostgreSQL, Redis, Qdrant, l'API et le Worker. La base de données est créée et initialisée automatiquement.

```
docker compose up -d
```

```
# Attendre 30 secondes puis vérifier :
```

```
docker compose ps
```

```
# Tous les services doivent être 'healthy' ou 'running'
```

5

Vérifier que l'API répond

Tester que le serveur fonctionne correctement.

```
# Test rapide :
```

```
curl http://localhost:8000/health
```

```
# Réponse attendue : {"status":"ok","app":"Marché-IA Cameroun"}
```

```
# Documentation interactive (navigateur) :
```

```
# http://localhost:8000/api/docs
```

```
# Interface base de données :
```

```
# http://localhost:8080
```

```
# Serveur: postgres | Login: marche_ia | Mdp: secret
```

6

Alimenter l'IA en connaissances (optionnel mais recommandé)

Le chatbot IA ne peut répondre précisément qu'après ingestion des textes officiels.

```
# Installer les dépendances Python localement :
```

```
pip install -r requirements.txt
```

```
# Placer les PDFs dans knowledge_base/ (voir knowledge_base/README.md)
```

```
# Lancer l'ingestion :
```

```
python scripts/ingest.py --source all
```

```
# L'opération dure 5 à 15 minutes selon la taille des PDFs
```

Lancer les tests automatiques

```
# Lancer les tests

# Installer les dépendances de test

pip install pytest pytest-asyncio pytest-mock httpx

# Lancer tous les tests

pytest tests/ -v --asyncio-mode=auto

# Résultat attendu :

# tests/test_backend.py::TestSecurity::test_hash_et_verify_password PASSED

# tests/test_backend.py::TestSecurity::test_jwt_create_et_decode PASSED

# tests/test_backend.py::TestHealth::test_health_ok PASSED

# ... (tous les tests doivent passer)

# Lancer un groupe de tests spécifique

pytest tests/ -v -k TestSecurity

pytest tests/ -v -k TestTenders
```

Vérifier la base de données

```
# Base de données

# Vérification automatique (40+ tests)

bash database/scripts/verify_database.sh

# Ouvrir psql directement

docker exec -it marche_ia_postgres psql -U marche_ia -d marche_ia_db

# Voir les tables

\d

# Voir la structure d'une table

\d tenders

# Compter les lignes

SELECT COUNT(*) FROM tenders WHERE status='open';

# Quitter psql

\q
```

PARTIE 12 — VARIABLES D'ENVIRONNEMENT

RÈGLE DE SÉCURITÉ ABSOLUE : Le fichier `.env` ne doit JAMAIS être partagé, committé sur Git, ou envoyé par email. Il est dans `.gitignore`. Si une clé API est exposée, la révoquer immédiatement.

Variable	Explication et comment l'obtenir
SECRET_KEY	Clé de signature JWT. Générer avec : <code>openssl rand -hex 32</code> . NE JAMAIS partager.
ANTHROPIC_API_KEY	Clé Claude API. Obtenir sur console.anthropic.com → API Keys.
DATABASE_URL	Connexion PostgreSQL. Format : <code>postgresql+asyncpg://user:pass@host:5432/dbname</code>
REDIS_URL	Cache Redis. Défaut : <code>redis://localhost:6379/0</code>
QDRANT_URL	Base vectorielle. Défaut : <code>http://localhost:6333</code>
LLM_MODEL	Modèle Claude. Défaut : <code>claude-sonnet-4-6</code>
LLM_TEMPERATURE	0.1 = réponses précises/factuelles. 0.7 = plus créatif. Garder 0.1 pour le juridique.
RAG_TOP_K	Nombre de chunks retournés par Qdrant. Défaut : 8. Augmenter si réponses incomplètes.
RAG_MIN_SCORE	Seuil de similarité minimum. Défaut : 0.65. Diminuer si trop de 'Je ne sais pas'.
CACHE_TTL_SECONDS	Durée de vie du cache Redis. Défaut : 3600 (1 heure).
ACCESS_TOKEN_EXPIRE_MINUTES	Durée du token JWT. Défaut : 1440 (24h). Réduire en production si nécessaire.
SMTP_HOST/USER/PASSWORD	Pour les alertes email. Utiliser un compte Gmail avec 'mot de passe d'application'.
WHATSAPP_TOKEN	Token Meta Cloud API. Obtenir via developers.facebook.com en créant une Business App.
WHATSAPP_PHONE_ID	ID du numéro WhatsApp Business. Visible dans le dashboard Meta Developers.
ARMP_FEED_URL	URL RSS de l'ARMP. Changer si l'URL officielle change.
SENTRY_DSN	Monitoring des erreurs. Gratuit sur sentry.io . Laisser vide en développement.
ENV	development staging production. Active Sentry et TrustedHostMiddleware en production.

PARTIE 13 — DIAGNOSTIC ET MAINTENANCE

Commandes Docker essentielles

```
# Docker

# Voir l'état de tous les conteneurs

docker compose ps

# Voir les logs en temps réel

docker compose logs -f api

docker compose logs -f worker

docker compose logs -f postgres

# Redémarrer un service spécifique

docker compose restart api

docker compose restart worker

# Arrêter proprement (données conservées)

docker compose down

# Arrêter ET supprimer toutes les données (IRRÉVERSIBLE)

docker compose down -v

# Reconstruire l'image après changement de code

docker compose build api && docker compose up -d api
```

Problèmes fréquents et solutions

Problème	Solution
API répond 503 sur /chat	Vérifier la clé ANTHROPIC_API_KEY dans .env. Vérifier que Qdrant est démarré. Voir les logs : docker compose logs api
Le chatbot répond 'Je ne trouve pas d'information'	La base de connaissances Qdrant est vide. Exécuter : python scripts/ingest.py --source all
docker compose up échoue sur PostgreSQL	Port 5432 déjà utilisé. Changer le port dans docker-compose.yml : '5433:5432'
Port 8000 déjà utilisé	Changer dans docker-compose.yml : '8001:8000' puis accéder sur http://localhost:8001
Les alertes WhatsApp ne s'envoient pas	Vérifier WHATSAPP_TOKEN et WHATSAPP_PHONE_ID dans .env. Le numéro destinataire doit être vérifié dans Meta Developers.

La sync ARMP échoue toutes les heures	Site ARMP peut être indisponible. Vérifier : docker compose logs worker. Le système réessaie automatiquement 3 fois.
Tests qui échouent	Vérifier la version Python (3.11+). Réinstaller : pip install -r requirements.txt. Vérifier pytest version : pip install pytest==8.2.2
Base de données vide après docker compose up	Les migrations s'exécutent au PREMIER démarrage seulement. Si la base existait déjà : bash database/scripts/reset_database.sh puis relancer.

Commandes de monitoring

```
# Monitoring

# Statistiques de la base de données

docker exec marche_ia_postgres psql -U marche_ia -d marche_ia_db -c \
"SELECT tablename, n_live_tup AS lignes FROM pg_stat_user_tables ORDER BY n_live_tup
DESC;"

# État de santé des syncs ARMP (vue SQL)

docker exec marche_ia_postgres psql -U marche_ia -d marche_ia_db -c \
"SELECT * FROM v_sync_health;"

# Métriques IA (tokens, latence, satisfaction)

docker exec marche_ia_postgres psql -U marche_ia -d marche_ia_db -c \
"SELECT * FROM v_ai_usage LIMIT 7;"

# Marchés urgents (deadline < 7 jours)

docker exec marche_ia_postgres psql -U marche_ia -d marche_ia_db -c \
"SELECT reference, title, days_remaining FROM v_tenders_urgent;"

# Cache Redis – statistiques

docker exec marche_ia_redis redis-cli info stats | grep hits

# Qdrant – nombre de vecteurs indexés

curl http://localhost:6333/collections/marche_ia_knowledge
```

Mise à jour du code en production

```
# Déploiement

# 1. Récupérer les nouveaux fichiers

git pull origin main

# 2. Reconstruire l'image Docker

docker compose build api worker
```

```
# 3. Redémarrer sans interruption (zero-downtime)
```

```
docker compose up -d --no-deps api worker
```

```
# 4. Vérifier que tout fonctionne
```

```
curl http://localhost:8000/health
```

```
pytest tests/ -v --asyncio-mode=auto
```